MODULE 05 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Lambda III – Event Sources and Failures

Three invocation models, three completely different retry stories – and the one question that decides which you are looking at.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 5 of 34

admin@sthithapragna.com

Before you can answer a Lambda failure question, you must know **how the function was invoked**. Everything else follows from that.

The three invocation models

MODEL	WHO CALLS IT, AND WHAT HAPPENS ON FAILURE
Synchronous	Caller waits for the response. Lambda does not retry. The caller must
Asynchronous	Lambda queues the event and returns 202 immediately. Lambda retries
Event source mapping	Lambda polls the source. The poller retries, and the source holds the data

ON THE EXAM

Synchronous: the caller owns the retry. Asynchronous: Lambda owns it. Event source mapping: the poller owns it, and the record survives in the source until it succeeds or expires.

Synchronous invocation

- API Gateway, Application Load Balancer, function URLs, direct **Invoke** calls.
- **Lambda performs no retries.** The error goes straight back to the caller.
- Dead-letter queues and destinations do **not** apply. There is nowhere to send it.
- Retry logic belongs in the client, with exponential backoff and jitter.
- A throttled synchronous call returns 429 immediately.

> ON THE EXAM
>
> If a question describes a synchronous trigger and asks how to capture failed events automatically, the honest answer is that you cannot - you handle it in the caller or change the model.

Asynchronous invocation

- S3 events, SNS, EventBridge, and **Invoke** with the Event invocation type.
- Lambda returns **202 Accepted** at once and queues the event internally.
- On a function error, Lambda retries **twice more**: after 1 minute, then after 2 minutes.
- The caller never learns the outcome. Success and failure look identical to it.
- So an unhandled failure is **silently discarded** unless you configure somewhere for it to go.

> ON THE EXAM
>
> Three attempts in total, not three retries. The silent-discard behaviour is why on-failure destinations exist.

Two kinds of failure, two retry behaviours

- **Function errors** – your code threw, or the invocation timed out. Retried **twice**, at 1 and 2 minutes.
- **Throttling (429) and system errors (5xx)** – the function never ran.
- Those are returned to the queue and retried for up to **6 hours**, with exponential backoff.
- Backoff grows from 1 second up to a maximum of 5 minutes between attempts.
- With a long queue, Lambda slows its own read rate and events can age out before they run.

> ON THE EXAM
>
> 'Retried twice' applies to code failures. A throttled event gets hours of attempts, because it never had a chance to run in the first place.

Controlling asynchronous retries

- **Maximum retry attempts:** 0, 1 or 2. Default 2.
- **Maximum event age:** 60 seconds to 6 hours. Default 6 hours.
- Whichever limit is reached first ends the attempts.
- Lower both when stale events are worse than lost ones - time-sensitive work, for example.
- Each retry is a full billed invocation. Retrying a doomed event three times costs three times.

> ON THE EXAM
>
> Set maximum event age low when processing an hour-old event would be wrong. A price update or a fraud check is worse late than never.

Dead-letter queue or destination?

DEAD-LETTER QUEUE

The original mechanism

- Failure only. No success path
- Targets SQS or SNS only
- Captures the event payload
- Error detail in message attributes
- Part of the function configuration

ON-FAILURE DESTINATION

What AWS recommends now

- Both onSuccess and onFailure
- Targets SQS, SNS, Lambda or EventBridge
- Captures the full invocation record
- Error message, stack trace and payloads
- Configured per version or alias

THE TIEBREAKER

Destinations are a superset. Choose them unless a question specifically describes a DLQ. When a scenario asks for the reason a failure happened, only destinations carry the stack trace.

What event source mappings are

- A Lambda-managed **poller** reads from the source and invokes your function with a batch.
- The **execution role** needs read permission on the source. No resource-based policy required.
- Records stay in the source until processed successfully, so nothing is lost mid-failure.
- Sources: SQS, Kinesis Data Streams, DynamoDB Streams, Amazon MSK, self-managed Kafka, Amazon MQ, DocumentDB.
- The mapping can be disabled without touching the function - a useful pause switch.

> ON THE EXAM
>
> This is the one model where permission flows the other way. The function reads the source, so the permission goes on the execution role, not on the function.

Batch size and batching window

- **Batch size** is the maximum records per invocation. SQS standard allows up to 10,000.
- **Maximum batching window** waits up to 300 seconds to fill a batch before invoking.
- Bigger batches mean fewer invocations, so lower cost and better throughput.
- Bigger batches also mean higher latency and a larger blast radius when one record fails.
- The invocation payload limit still applies, so a batch may arrive smaller than configured.

> ON THE EXAM
>
> Batching trades latency for cost. A low-traffic queue with a large batch size and no batching window can leave records waiting a long time.

SQS as an event source

- Successfully processed messages are **deleted by Lambda** when the invocation succeeds.
- On failure, nothing is deleted. The whole batch reappears after the visibility timeout.
- Retries are governed by the queue's **maxReceiveCount**, not by Lambda.
- The **DLQ belongs to the queue**, configured in its redrive policy.
- A Lambda DLQ does nothing here. That is a function-level asynchronous setting.

> ON THE EXAM
>
> The last point is a classic distractor. For an SQS-triggered function, the dead-letter queue is configured on the queue, not on the function.

The visibility timeout rule

- AWS recommends **visibility timeout = 6 x function timeout + MaximumBatchingWindowInSeconds**.
- Too short, and the message reappears while your function is still working on it.
- A second invocation then processes the same message. That is duplicate processing.
- The 6x allowance exists to leave room for retries after a throttle.
- The SQS default visibility timeout is 30 seconds - shorter than many function timeouts.

> ON THE EXAM
>
> Reports of messages being processed twice, with no errors in the logs, point at the visibility timeout before they point at anything else.

Partial batch responses

- By default one bad record fails the whole batch, so **successful records are reprocessed too**.
- Set **ReportBatchItemFailures** in FunctionResponseTypes on the event source mapping.
- The function then returns a list of failed item identifiers, and only those become visible again.
- If the function **throws** instead of returning, the entire batch still fails.
- The Powertools Batch Processor utility implements this for you.

> ON THE EXAM
>
> This is the standard fix for 'nine records succeeded, one failed, and all ten ran again'. It needs both the mapping setting and a code change.

Starting position and iterator age

- **TRIM_HORIZON** starts at the oldest record still in the stream.
- **LATEST** starts just after the newest record, so existing data is skipped.
- **AT_TIMESTAMP** starts from a point in time you supply.
- Creating or updating a mapping is eventually consistent, so **LATEST can miss events** during the change.
- **IteratorAge** is the age of the last record processed. Rising means the consumer is falling behind.

> ON THE EXAM
>
> Iterator age climbing steadily is the single clearest sign of a struggling stream consumer. It never applies to SQS, which has no ordered position.

Streams behave differently

- Records are **ordered within a shard**, and Lambda preserves that order.
- By default **one concurrent invocation per shard**. Concurrency is shard count, not traffic.
- A failing batch is **retried until it succeeds or the records expire** - blocking that shard.
- That is the poison-pill problem. One bad record can stall a shard for its whole retention period.
- Records are not deleted after processing. They stay until retention expires.

> ON THE EXAM
>
> Ordering is the reason for the blocking. Lambda cannot skip a record without breaking the guarantee it is giving you.

Escaping a poison pill

- **BisectBatchOnFunctionError** splits a failed batch in half repeatedly to isolate the bad record.
- **MaximumRetryAttempts** caps retries so the shard eventually moves on.
- **MaximumRecordAgeInSeconds** discards records older than a threshold.
- **On-failure destination** sends metadata about skipped records to SQS or SNS.
- Without a destination configured, skipped records are dropped and gone.

ON THE EXAM

Bisecting narrows the blast radius. Maximum retries and record age unblock the shard. The destination is what stops you losing the data while doing it.

Parallelisation factor

- Raises concurrent invocations **per shard**, from 1 up to 10.
- Ordering is still preserved **within each partition key**, so the guarantee survives.
- The cheaper way to scale a stream consumer than resharding.
- It does not help if the bottleneck is a downstream system rather than Lambda.
- Applies to Kinesis Data Streams and DynamoDB Streams.

> ON THE EXAM
>
> When a scenario says the consumer cannot keep up and iterator age is climbing, this is usually cheaper and faster than adding shards.

Tumbling windows

- Fixed time windows, **up to 15 minutes**, that let state pass between invocations.
- The event carries **window**, **state** and **isFinalInvokeForWindow**.
- Lets you aggregate across batches without an external database.
- State is capped at **1 MB per shard**. Exceed it and the window is terminated early.
- Not supported across resharding. A closing shard ends its window.

> ON THE EXAM
>
> The usual answer for running totals over a stream is DynamoDB or ElastiCache. Tumbling windows are the answer when the aggregation fits in a window and 1 MB.

Event filtering

- **Filter criteria** on the event source mapping discard non-matching records **before invocation**.
- You are not billed for events that are filtered out. There is no invocation.
- Available for SQS, Kinesis, DynamoDB Streams, MSK, Kafka and DocumentDB.
- Up to 5 filter patterns per mapping, using EventBridge pattern syntax.
- Filtering in your handler still costs a full invocation. The saving comes from filtering earlier.

> ON THE EXAM
>
> When a scenario says a function is invoked constantly but discards most events, this is the intended answer - lower cost and no code change.

S3 events: direct or via EventBridge

- **S3 event notifications** invoke Lambda directly. Simple, and asynchronous.
- Only one destination per event type and prefix combination. Fan-out needs SNS in between.
- **S3 to EventBridge** emits every event to the default bus instead.
- EventBridge gives many targets, richer filtering, archive and replay.
- It also covers more event types than notifications do.

> ON THE EXAM
>
> One consumer, keep it simple: notifications. Several consumers, or you want filtering and replay: EventBridge.

Every retry mechanism in this module can deliver the same event twice. Lambda gives you **at-least-once**, never exactly-once.

Building idempotency

- Pick a stable **idempotency key**: an order ID or message ID, not a timestamp or a random value.
- Record it in DynamoDB with a **conditional write** so a duplicate fails the condition.
- Use **TTL** on those records so the table does not grow without bound.
- The **Powertools idempotency utility** implements the whole pattern as a decorator.
- Naturally idempotent operations need nothing. Setting a status to 'shipped' twice is harmless.

> ON THE EXAM
>
> The distinction to make quickly: does running this twice do damage? Charging a card, yes. Overwriting a record with the same value, no.

Recursive loop detection

- The classic accident: a function triggered by an S3 bucket that writes back to the same bucket.
- Lambda tracks a **Lineage** counter in the X-Ray trace header across the chain.
- After roughly **16** invocations of the same event chain, Lambda stops the next invocation.
- It emits **RecursiveInvocationsDropped** and notifies you by email and Health Dashboard.
- On by default, covering Lambda, SQS, SNS and S3. Requires a supported SDK version.

> ON THE EXAM
>
> It limits the damage, it does not fix the design. If you hit it, set reserved concurrency to zero, then fix the loop.

Failure handling, by source

TRIGGER	WHERE FAILURES GO
API Gateway, ALB, function URL	Nowhere. The caller sees the error and must retry
S3, SNS, EventBridge	Lambda DLQ or on-failure destination, after 2 retries
SQS	The queue's own DLQ, via maxReceiveCount in the redrive policy
Kinesis, DynamoDB Streams	On-failure destination, after retry and record-age limits

ON THE EXAM

One table, four rows, and a large slice of the troubleshooting domain. The SQS row is the one most often answered wrongly.

EventBridge Pipes

- A managed point-to-point integration: **source, filter, enrichment, target**.
- Sources include SQS, Kinesis, DynamoDB Streams, MSK and Amazon MQ.
- Removes the Lambda function that existed only to read from one service and write to another.
- Filtering and simple transformation happen without any code at all.
- Real business logic still belongs in a function. Pipes is for plumbing, not processing.

> ON THE EXAM
>
> When a scenario describes a function that only reads from a queue and forwards to another service, and asks for the least operational overhead, Pipes is often the intended answer.

SQS or Kinesis for this consumer?

CHOOSE SQS WHEN

Independent work items

- Order does not matter
- Each message is consumed once
- You want concurrency to follow load
- One bad message must not block others
- Simple retry and DLQ semantics

CHOOSE KINESIS WHEN

An ordered record stream

- Order within a key matters
- Several consumers read the same data
- You may need to replay history
- Concurrency is bounded by shards
- A bad record can block a shard

THE TIEBREAKER

Ordering and replay push you to Kinesis. Independent tasks and non-blocking failures push you to SQS. 'Multiple consumers of the same events' is the clearest single signal for Kinesis.

For each one, identify the **invocation model** first. Then decide.

> An order-processing Lambda function reads from an Amazon SQS standard queue with a batch size of 10. It charges a payment provider for each order in the batch.
>
> Customers report occasional duplicate charges. Investigation shows that when one message in a batch fails, all ten become visible again and the whole batch is reprocessed. The failing message is usually a genuine data problem affecting a single order.

What combination best prevents the duplicate charges?

A. Reduce the batch size to 1 so a failure can only ever affect one message

B. Increase the queue's visibility timeout to six times the function timeout

C. Configure a Lambda on-failure destination to capture the failed batch

D. Enable ReportBatchItemFailures on the event source mapping, return the failed message IDs, and make the charge operation idempotent using the order ID

◆ PAUSE AND DECIDE

What combination best prevents the duplicate charges?

A Batch size of 1

It would work, at ten times the invocations and ten times the cost, and it throws away batching entirely. A blunt instrument for a solved problem.

B Longer visibility timeout

Fixes a different duplicate-processing cause - messages reappearing mid-execution. Here the batch is failing outright, not timing out.

C On-failure destination

Destinations are for asynchronous invocation. This is an event source mapping, so this setting does not apply.

D Partial batch responses plus idempotency

Stops the nine good messages being returned to the queue, and idempotency protects against the duplicates that at-least-once delivery can still produce.

THE SIGNAL

Two defences, deliberately. Partial batch responses remove the cause; idempotency covers every other retry path that could still deliver twice.

A Lambda function consumes a Kinesis Data Stream carrying device telemetry. One device began emitting a malformed record, and the function throws on it. Processing for that shard has stopped completely, and iterator age has been climbing for hours.

The team needs processing to resume, wants the surrounding good records processed rather than discarded, and must keep any skipped records for later investigation.

Which configuration meets all three requirements?

A Increase the number of shards in the stream

B Enable BisectBatchOnFunctionError, set MaximumRetryAttempts, and configure an on-failure destination

C Set the parallelisation factor to 10

D Configure a dead-letter queue on the Lambda function

◆ PAUSE AND DECIDE

Which configuration meets all three requirements?

A More shards

Resharding does not remove the bad record. That shard's records are redistributed but the poison pill still blocks whichever shard holds it.

B Bisect, retry limit, and destination

Bisecting isolates the bad record so good ones proceed, the retry cap unblocks the shard, and the destination preserves what was skipped. All three requirements.

C Parallelisation factor

Adds throughput, not fault tolerance. The blocked partition key is still blocked, so nothing here resumes processing.

D Lambda DLQ

A function-level asynchronous setting. Stream event source mappings use an on-failure destination instead.

THE SIGNAL

Three requirements, three settings. Only B addresses all of them, and the third one - keeping the skipped records - is what eliminates everything else.

A Lambda function is triggered by S3 object-created events to generate thumbnails. Occasionally an upload produces no thumbnail and nobody notices for days. There is no record of what happened.

The team wants every permanent failure captured automatically, including the error message and stack trace, so a separate function can attempt recovery and raise an alert. They do not want to modify the thumbnail function's code.

What should the team configure?

A. An on-failure destination pointing at a Lambda function that handles recovery and alerting

B. A dead-letter queue on the function, pointing at an SQS queue

C. Wrap the handler in a try/except block that publishes failures to SNS

D. Enable ReportBatchItemFailures on the S3 event source mapping

◆ PAUSE AND DECIDE

What should the team configure?

A On-failure destination to a Lambda function

Destinations carry the full invocation record including the stack trace, and can target a Lambda function directly. No code change needed.

B Dead-letter queue to SQS

The closest wrong answer. It captures the event, but only limited error detail in message attributes, and it cannot invoke a function - something must poll the queue.

C try/except publishing to SNS

Explicitly ruled out, since the team will not modify the code. It also misses timeouts and out-of-memory kills, which never reach an exception handler.

D ReportBatchItemFailures

S3 invokes Lambda asynchronously. There is no event source mapping here, so the setting does not exist to enable.

THE SIGNAL

Two constraints do the work: the stack trace is needed, and the code cannot change. Those together select destinations over a DLQ.

A Lambda function is attached to a DynamoDB Stream on a large table. It exists solely to send a notification when an order's status changes to CANCELLED. Status changes to CANCELLED are about 2% of all writes.

The function is invoked roughly 40 million times a month. Its first line inspects the record and returns immediately for anything that is not a cancellation. The Lambda bill has become a problem and the team wants it reduced without changing behaviour.

What is the most effective change?

A Reduce the function's memory to 128 MB so each invocation costs less

B Increase the batch size so fewer invocations are needed

C Add filter criteria to the event source mapping so only CANCELLED records invoke the function

D Move the status check into a Step Functions Choice state before the Lambda function

◆ PAUSE AND DECIDE

What is the most effective change?

A Lower the memory

Shaves a little from each invocation while still paying for all 40 million. It also risks making the function slower.

B Larger batches

A real improvement, and the second-best answer. But it still delivers every record to the function and still pays per request.

C Filter criteria on the event source mapping

Non-matching records are discarded before invocation, so 98% of those invocations stop happening and stop being billed. No code change.

D Step Functions Choice state

Adds a state machine and its own per-transition charge to every event. It makes the bill worse while adding a component.

THE SIGNAL

The cheapest invocation is the one that never happens. Filtering at the event source is the only option that removes invocations rather than making them cheaper.

What to carry into Lambda IV

- **Identify the invocation model first.** It determines everything about failure handling.
- Async retries twice on function errors, but up to 6 hours on throttles and system errors.
- **SQS failures go to the queue's DLQ**, never to a Lambda DLQ.
- Streams block on failure. Bisect, cap retries, and add an on-failure destination.
- Delivery is **at-least-once**. Idempotency is a requirement, not a refinement.

> ON THE EXAM
>
> Next: response streaming, function URLs, edge compute, and the new Lambda capabilities that are changing what belongs in a function at all.